

// Day 12 of 30

DEEP DIVE

Java HashMap Internals — How It Actually Works

→ Swipe through all 6 slides



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

1/8

What HashMap Really Is

Not just key-value. It's an array of linked lists.

```
HashMap map = new HashMap<>();
```

Internally:

```
Node[] table;  
// Array of 16 buckets (default)  
// Each bucket = linked list/tree
```

💡 Default capacity: 16, Load factor: 0.75



How put() Works

4 steps every time you insert

```
map.put("name", 42);
```

Step 1: `hash("name")` → hashCode

Step 2: `index = hash & (n-1)`
→ bucket position

Step 3: Check bucket

Empty? → insert Node

Step 4: Not empty? → check `equals()`
Same key? → overwrite
Different? → add to chain

💡 `hashCode()` decides bucket, `equals()` decides match



How get() Works

Same logic, reverse direction

```
map.get("name");
```

Step 1: `hash("name")` → `hashCode`

Step 2: `index = hash & (n-1)`

Step 3: Go to `bucket[index]`

Step 4: Compare keys using `equals()`

Found? → `return value`

Not found? → `return null`

💡 $O(1)$ average, $O(n)$ worst case



What Happens at 75% Full

Rehashing — the expensive operation

When `size > capacity * 0.75`:

`16 * 0.75 = 12` entries triggers resize

Resize process:

1. Create new array (32 buckets)
2. Rehash EVERY existing entry
3. Recompute index for each
4. Move to new positions

// This is $O(n)$ – expensive!

💡 Set initial capacity if you know the size



Treeification (Java 8+)

When a bucket gets too long

If one bucket has 8+ entries:

Linked List $\rightarrow$ Red-Black Tree

$O(n)$ lookup $\rightarrow O(\log n)$ lookup

If bucket shrinks below 6:

Tree $\rightarrow$ back to Linked List

// Protects against hash collision attacks

💡 Java 8 made HashMap much more resilient



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

Interview Killer Questions

You'll face these. Now you're ready.

Q: Why capacity is always power of 2?

A: For fast modulo: $\text{hash} \& (n-1)$

Q: Why load factor is 0.75?

A: Balance between space and speed

Q: What if two keys have same hash?

A: Collision $\rightarrow$ same bucket $\rightarrow$ equals()

Q: Is HashMap thread-safe?

A: No. Use ConcurrentHashMap.

💡 These 4 questions = 90% of HashMap interviews



done = true

Follow for more!

Like · Comment · Repost

Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)